

Q1 (a) — Test Plan

1. Objective of Testing

To verify and validate that the InsightFlow platform functions correctly across all modules — authentication, dataset management, AI chat (Gemini-powered RAG), report generation, and the frontend UI. Testing aims to ensure reliability, data integrity, security, and a smooth user experience.

2. Scope — Modules / Features to be Tested

Module	Features
Authentication	Login, Register, Token handling, Session management
Dataset Management	CSV/Excel upload, File listing, File deletion, Data analysis
AI Chat (RAG Pipeline)	General chat via Gemini API, PDF upload & indexing, PDF Q&A; with context retrieval, TF-IDF vectorization
Report Generation	Report creation from datasets, Report listing
Frontend UI	Page routing, Form validations, Responsive design, Dashboard rendering
API Endpoints	Health check, All REST API routes, Error handling

3. Types of Testing to be Performed

Type	Description
Unit Testing	Individual functions — auth controller, data analyzer, RAG engine query/chunking
Integration Testing	API routes + controllers, RAG pipeline (PDF → chunks → TF-IDF → Gemini)
System Testing	End-to-end flows — register → login → upload → chat → report
Functional Testing	Each feature against expected behavior
Security Testing	Auth middleware, token validation, input sanitization
UI/UX Testing	Form validation, navigation, responsive layout, error messages

4. Tools

Tool	Purpose
cURL	Backend API testing
Browser DevTools	Frontend debugging, network inspection
Vite HMR	Live frontend testing during development
Python unittest	Server-side unit tests
Flask Test Client	Integration testing of routes

5. Entry and Exit Criteria

Entry Criteria:

- Source code is complete and deployable
- Backend server (Flask) starts without errors on port 5001
- Frontend dev server (Vite) starts without errors on port 5173
- All dependencies installed (pip install -r requirements.txt, npm install)

Exit Criteria:

- All critical and high-severity test cases pass
- No critical bugs remain unresolved
- At least 90% of test cases pass
- All API endpoints return correct status codes
- UI renders correctly on the target browser

Q1 (b) — Test Cases for the Authentication & API Module

TC ID	Scenario	Input	Expected	Actual Output	Status
TC-01	Server health check	GET /health	{"status":"healthy"} HTTP 200	{"status":"healthy"} HTTP 200	Pass
TC-02	Login with valid credentials	POST /api/auth/login {"email":"test@example.com", "password":"pass123"}	HTTP 200 with token and user object	{"message":"Login successful", "token":"mock-jwt-token-123", "user":{"...}} HTTP 200	Pass
TC-03	Login with empty email and password	POST /api/auth/login {"email":"","password":""}	HTTP 400 with error message	{"error":"Missing credentials"} HTTP 400	Pass
TC-04	Register a new user	POST /api/auth/register {"email":"new@test.com", "name":"Test User", "password":"secure123"}	HTTP 201 with user object	{"message":"User registered successfully", "user":{"...}} HTTP 201	Pass
TC-05	List datasets (authenticated)	GET /api/datasets/ Authorization: Bearer mock-token	HTTP 200 with datasets array	{"datasets":[], "status":200} HTTP 200	Pass
TC-06	Upload dataset without file	POST /api/datasets/upload — no file attached	HTTP 400 — "No file part"	{"error":"No file part"} HTTP 400	Pass
TC-07	Generate report without dataset_id	POST /api/reports/generate — body: {}	HTTP 400 — "Dataset ID required"	{"error":"Dataset ID required"} HTTP 400	Pass
TC-08	Chat query with valid question	POST /api/chat/query {"query":"What is AI?", "document_id":"doc1"}	HTTP 200 with answer and sources	{"answer":"...automated response...", "sources":["Page 1","Page 2"]} HTTP 200	Pass

Q2 (a) — Test Execution Results with Evidence

All 8 test cases were executed against the running Flask backend at <http://localhost:5001> using cURL commands.

Test TC-01: Health Check

```
$ curl -s http://localhost:5001/health {"status": "healthy"}
```

Result: PASS — Server responds correctly.

Test TC-02: Login with Valid Credentials

```
$ curl -s -X POST http://localhost:5001/api/auth/login \ -H "Content-Type: application/json" \ -d '{"email": "test@example.com", "password": "pass123"}' {"message": "Login successful", "token": "mock-jwt-token-123", "user": {"email": "test@example.com", "id": "e32849cc-..."}}
```

Result: PASS — Returns token and user object with HTTP 200.

Test TC-03: Login with Empty Fields

```
$ curl -s -X POST http://localhost:5001/api/auth/login \ -H "Content-Type: application/json" \ -d '{"email": "", "password": ""}' {"error": "Missing credentials"}
```

Result: PASS — Correctly rejects empty credentials with error.

Test TC-04: Register New User

```
$ curl -s -X POST http://localhost:5001/api/auth/register \ -H "Content-Type: application/json" \ -d '{"email": "new@test.com", "name": "Test User", "password": "secure123"}' {"message": "User registered successfully", "user": {"email": "new@test.com", "id": "120437f7-...", "name": "Test User"}}
```

Result: PASS — User registered with unique UUID.

Test TC-05: List Datasets

```
$ curl -s http://localhost:5001/api/datasets/ \ -H "Authorization: Bearer mock-token" {"datasets": [], "status": 200}
```

Result: PASS — Returns empty dataset list as expected.

Test TC-06: Upload Without File

```
$ curl -s -X POST http://localhost:5001/api/datasets/upload \ -H "Authorization: Bearer mock-token" {"error": "No file part"}
```

Result: PASS — Correctly rejects request with missing file.

Test TC-07: Generate Report Without Dataset ID

```
$ curl -s -X POST http://localhost:5001/api/reports/generate \ -H "Content-Type: application/json" \ -H "Authorization: Bearer mock-token" \ -d '{}' {"error": "Dataset ID required"}
```

Result: PASS — Correctly validates required field.

Test TC-08: Chat Query

```
$ curl -s -X POST http://localhost:5001/api/chat/query \ -H "Content-Type: application/json" \ -H "Authorization: Bearer mock-token" \ -d '{"query": "What is AI?", "document_id": "doc1"}' {"answer": "This is an automated response to 'What is AI?' for document doc1.", "sources": ["Page 1", "Page 2"]}
```

Result: PASS — Returns answer and sources.

Summary

Total Test Cases	Passed	Failed
8	8	0

All 8 test cases passed successfully against the running backend server.

Q2 (b) — Defect (Bug) Reports

Bug #1: Authentication Middleware Bypass — Protected Routes Accessible Without Token

Bug ID	BUG-001
Description	Protected API routes (datasets, reports) are accessible without any authentication token. The require_auth middleware does not enforce token validation — it allows all requests through regardless of the Authorization header.
Steps to Reproduce	1. Start the backend server. 2. Send a GET request without any Authorization header: curl -s http://localhost:5001/api/datasets/ 3. Observe the response.
Expected Result	HTTP 401 Unauthorized — {"error": "Authentication required"}
Actual Result	HTTP 200 OK — {"datasets":[], "status":200} — Full access granted without any token.
Severity	HIGH — Any unauthenticated user can access, upload, or delete datasets and generate reports. This is a critical security vulnerability.
Suggested Fix	Modify server/middleware/authMiddleware.py to verify the JWT token and return HTTP 401 if the token is missing or invalid: if not auth_header or not auth_header.startswith('Bearer '): return jsonify({'error': 'Authentication required'}), 401

Evidence:

```
$ curl -s http://localhost:5001/api/datasets/ {"datasets":[], "status":200} # No Authorization header sent, yet access was granted
```

Bug #2: Login Accepts Any Password — No Credential Verification

Bug ID	BUG-002
Description	The login endpoint accepts ANY email/password combination without verifying credentials against stored data. Any non-empty password returns "Login successful" with a hardcoded mock token.
Steps to Reproduce	1. Send a login request with a clearly wrong password: curl -s -X POST http://localhost:5001/api/auth/login \ -H "Content-Type: application/json" \ -d '{"email":"test@example.com","password":"wrong_password"}' 2. Observe that login succeeds.
Expected Result	HTTP 401 — {"error": "Invalid email or password"}
Actual Result	{"message":"Login successful","token":"mock-jwt-token-123","user":{"...}} HTTP 200
Severity	HIGH — Any attacker can log in as any user with any password. Complete authentication bypass.
Suggested Fix	Integrate a database (e.g., MongoDB) to store user credentials with hashed passwords (bcrypt). On login, look up the user by email and verify the password hash before issuing a JWT token.

Evidence:

```
$ curl -s -X POST http://localhost:5001/api/auth/login \ -H "Content-Type: application/json" \ -d '{"email":"test@example.com","password":"wrong_password"}' {"message":"Login successful","token":"mock-jwt-token-123", "user":{"email":"test@example.com","id":"589eb192-..."}} # Login succeeded with a completely wrong password
```

Bug #3: Duplicate Registration Allowed — No Unique Email Constraint

Bug ID	BUG-003
--------	---------

Description	The register endpoint allows the same email to be registered multiple times. Each registration creates a new user with a different UUID, without checking if the email already exists.
Steps to Reproduce	1. Register with email same@test.com (returns HTTP 201). 2. Register again with the same email and a different name/password. 3. Both requests return HTTP 201 success.
Expected Result	Second registration should fail with HTTP 409 — {"error": "Email already registered"}
Actual Result	Both succeed — two different users created with the same email but different UUIDs.
Severity	MEDIUM — Causes data inconsistency. Multiple accounts can exist for the same email, leading to confusion in login and data ownership.
Suggested Fix	Before creating a user, query the database to check if the email already exists. If it does, return HTTP 409 Conflict with an appropriate error message.

Evidence:

```
# First registration $ curl -s -X POST http://localhost:5001/api/auth/register \ -d
'{"email":"same@test.com","name":"User1","password":"pass1"}'
{"user":{"email":"same@test.com","id":"17d26486-...", "name":"User1"}} # Second registration – SAME
email $ curl -s -X POST http://localhost:5001/api/auth/register \ -d
'{"email":"same@test.com","name":"User2","password":"pass2"}'
{"user":{"email":"same@test.com","id":"cfcfe435-...", "name":"User2"}} # Both succeeded – duplicate
accounts created
```

Defect Summary

Bug ID	Description	Severity	Status
BUG-001	Auth middleware allows unauthenticated access to protected routes	High	Open
BUG-002	Login accepts any password without verification	High	Open
BUG-003	Duplicate email registration allowed	Medium	Open